



ISGA

Filière Ingénieur — Système Informatique

Cyber Copilot

Plateforme d'investigation de sécurité assistée par intelligence
artificielle,
bâtie sur Wazuh

Rapport de Projet de Fin d'Année

4^{ème} année — Cycle Ingénieur

Réalisé par :

Abdelkrim El Ayachi

Yassin El Makdour

Badr Alammague

Encadré par :

Adama Samake

Année universitaire : 2026 – 2027

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude à notre encadrant, **M. Adama Samake**, pour sa disponibilité, ses conseils avisés et le suivi rigoureux qu'il a accordé à ce projet tout au long de l'année. Sa confiance et ses orientations méthodologiques ont été déterminantes dans la réussite de ce travail.

Nous remercions également l'ensemble du corps professoral et administratif de **l'ISGA** pour la qualité de la formation dispensée durant ces années d'études, qui nous a permis d'acquérir les compétences techniques et méthodologiques mobilisées dans ce projet.

Enfin, nous souhaitons remercier les membres du jury qui nous font l'honneur d'évaluer ce travail, ainsi que toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet.

Dédicaces

*À nos chers parents, pour leur soutien inconditionnel, leurs sacrifices
et leurs encouragements tout au long de notre parcours académique.*

*À nos familles et nos amis, pour leur patience et leur présence
dans les moments de doute comme dans ceux de réussite.*

*À tous ceux qui croient que la technologie, utilisée avec discernement,
peut rendre le monde numérique plus sûr.*

Liste des figures

1.1	Diagramme de Gantt du projet	3
3.1	Architecture globale de Cyber Copilot	8
3.2	Diagramme de cas d'utilisation	9
3.3	Diagramme de classes (extrait représentatif)	10
3.4	Séquence – investigation automatique d’une alerte	10
3.5	Séquence – authentification d’un utilisateur	11
3.6	Modèle de données simplifié (MySQL)	11

Liste des tables

2.1	Comparatif entre les approches existantes et Cyber Copilot	7
3.1	Choix technologiques et justifications	12
4.1	Outils exposés à l'IA	17
5.1	Principales validations fonctionnelles effectuées	20
A.1	Principales variables d'environnement	27

Liste des abréviations

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DDL	Data Definition Language
ERD	Entity-Relationship Diagram (diagramme entité-association)
FPS	Faux Positifs (alertes de sécurité non pertinentes)
HTTP(S)	HyperText Transfer Protocol (Secure)
IA	Intelligence Artificielle
IDS	Intrusion Detection System
IOC	Indicator of Compromise (indicateur de compromission)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (grand modèle de langage)
MITRE ATT&CK	Base de connaissances de tactiques et techniques d'attaquants (MITRE Corporation)
ORM	Object-Relational Mapping
OSINT	Open Source Intelligence
REST	Representational State Transfer
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SPA	Single Page Application
SQL	Structured Query Language
TPM	Tokens Per Minute (quota de débit d'un fournisseur de LLM)
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XDR	Extended Detection and Response

Sommaire

Remerciements	i
Dédicaces	ii
Liste des figures	iii
Liste des tables	iv
Liste des abréviations	v
Sommaire	vi
1 Introduction générale	1
1.1 Contexte général	1
1.2 Problématique	1
1.3 Objectifs du projet	2
1.4 Méthodologie de gestion du projet	2
1.5 Étude de faisabilité	3
1.6 Plan du rapport	4
2 État de l’art	5
2.1 Gestion des incidents de sécurité et SIEM	5
2.2 Wazuh comme plateforme SIEM/XDR open source	5
2.3 Fatigue d’alerte et automatisation (SOAR)	6
2.4 Agents fondés sur les grands modèles de langage	6
2.5 Comparatif des approches existantes	6
2.6 Synthèse et positionnement du projet	7
3 Conception	8
3.1 Architecture globale du système	8
3.2 Diagramme de cas d’utilisation	8
3.3 Diagramme de classes	9
3.4 Diagrammes de séquence	10
3.4.1 Investigation automatique d’une alerte	10
3.4.2 Authentification d’un utilisateur	10
3.5 Modèle de données	11
3.6 Choix technologiques	12
3.7 Sécurité de la conception	12

4	Implémentation / Réalisation	14
4.1	Environnement de travail	14
4.1.1	Environnement logiciel	14
4.1.2	Environnement matériel	14
4.2	Description et traitement des données	14
4.3	Modules principaux	15
4.3.1	Collecte et politiques (<code>poller_service.py</code> , <code>investigation_policy.py</code>)	15
4.3.2	Gestion des investigations (<code>investigation_manager.py</code>)	15
4.3.3	Agent d'investigation (<code>ai_investigator/investigator.py</code>) . .	15
4.3.4	Outils disponibles pour l'IA	17
4.3.5	Assistant conversationnel (<code>ai_investigator/chatbot.py</code>) . . .	17
4.3.6	Authentification (<code>api/auth.py</code>)	17
4.4	Interface utilisateur	18
5	Présentation des résultats	19
5.1	Scénario de bout en bout	19
5.2	Validations fonctionnelles effectuées	19
5.3	Limites constatées	20
5.4	Discussion	21
6	Conclusion générale et perspectives	22
6.1	Synthèse du travail réalisé	22
6.2	Difficultés rencontrées	22
6.3	Perspectives	23
	Références	24
A	Annexes	25
A.1	Structure du dépôt de code	25
A.2	Extrait du schéma de la table <code>investigations</code>	26
A.3	Extrait de l'invite système de l'agent d'investigation	26
A.4	Variables de configuration principales (<code>.env</code>)	27
A.5	Glossaire complémentaire	27

Chapter 1

Introduction générale

1.1 Contexte général

La multiplication des cybermenaces et la sophistication croissante des attaques informatiques ont conduit la majorité des organisations à se doter de systèmes de détection et de supervision de sécurité, communément appelés **SIEM** (*Security Information and Event Management*). Ces plateformes centralisent les journaux (*logs*) issus des différents équipements du système d'information — postes de travail, serveurs, équipements réseau — et génèrent des **alertes** lorsqu'un comportement suspect est détecté.

Dans ce projet, nous nous appuyons sur **Wazuh**, une plateforme SIEM/XDR open source largement adoptée, qui combine un agent de collecte installé sur les hôtes surveillés, un moteur de règles de détection, et un moteur de recherche (*Wazuh Indexer*, basé sur OpenSearch) qui stocke et indexe les alertes générées.

Le problème central que rencontrent les équipes de sécurité opérationnelle (**SOC**) n'est cependant pas la détection elle-même, mais le **volume** d'alertes à traiter quotidiennement. Une étude régulièrement citée dans l'industrie (rapports annuels de fournisseurs SIEM et enquêtes SANS sur les opérations SOC) montre qu'un analyste peut être confronté à plusieurs centaines, voire plusieurs milliers, d'alertes par jour, dont une large proportion sont des **faux positifs**. Ce phénomène, connu sous le nom de *fatigue liée aux alertes* (*alert fatigue*), dégrade la capacité de détection réelle des menaces : les analystes finissent par traiter les alertes de manière superficielle, voire par les ignorer.

1.2 Problématique

Comment réduire la charge de triage manuel des alertes de sécurité tout en conservant — voire en améliorant — la qualité et la traçabilité des décisions prises ? Plus précisément, ce projet cherche à répondre aux questions suivantes :

- Peut-on automatiser la phase d'**investigation initiale** d'une alerte (collecte de contexte, recherche d'alertes corrélées, vérification de la réputation d'une adresse IP, etc.) à l'aide d'un agent d'intelligence artificielle ?
- Un tel agent peut-il produire un **verdict argumenté** (*vrai positif, faux positif, ou à revoir*) avec un niveau de confiance et une justification exploitable par un humain ?

- Comment garantir que cet agent reste **fiable, traçable et borné** (en coût, en temps, en accès aux données) alors qu'il s'appuie sur un modèle de langage dont le comportement n'est pas entièrement déterministe ?
- Comment restreindre l'accès de l'agent aux données sensibles (identités des utilisateurs, par exemple) sans complexifier son architecture ?

1.3 Objectifs du projet

L'objectif général du projet est de concevoir et développer **Cyber Copilot**, une plateforme web qui :

1. collecte automatiquement les alertes générées par Wazuh (*polling* périodique sur le Wazuh Indexer) ;
2. détermine, à l'aide d'un **moteur de politiques** configurable, quelles alertes méritent l'ouverture d'une investigation ;
3. confie chaque investigation à un **agent IA** (*AI Investigator*) qui raisonne de manière itérative (*boucle agentique*) : il consulte des outils (recherche d'alertes, historique des investigations, réputation d'IP, informations sur les actifs), accumule des preuves, puis rend un verdict argumenté ;
4. enregistre **chaque action** de l'agent (outil appelé, raisonnement, résultat) pour permettre un audit humain complet ;
5. propose un **assistant conversationnel** (chatbot) partageant les mêmes outils, pour interroger librement la plateforme en langage naturel ;
6. sécurise l'accès à la plateforme par une **authentification** déléguée aux comptes Wazuh existants, évitant la duplication d'une base de mots de passe ;
7. expose l'ensemble via une **interface web** (tableau de bord, liste et détail des investigations, gestion des actifs et des politiques, chat).

1.4 Méthodologie de gestion du projet

Le projet a été mené selon une démarche **itérative et incrémentale**, proche des principes agiles : chaque brique fonctionnelle (collecte d'alertes, moteur de politiques, agent d'investigation, interface web, assistant conversationnel, authentification) a été conçue, développée puis validée indépendamment avant intégration, ce qui a permis d'obtenir à tout moment une chaîne fonctionnelle de bout en bout, même partielle.

Le déroulement du projet a suivi les grandes phases imposées par le cadre pédagogique du Projet de Fin d'Année : définition du sujet, étude de l'existant, conception, implémentation, présentation des résultats, et rédaction du rapport. Le diagramme de Gantt ci-dessous synthétise cette planification.

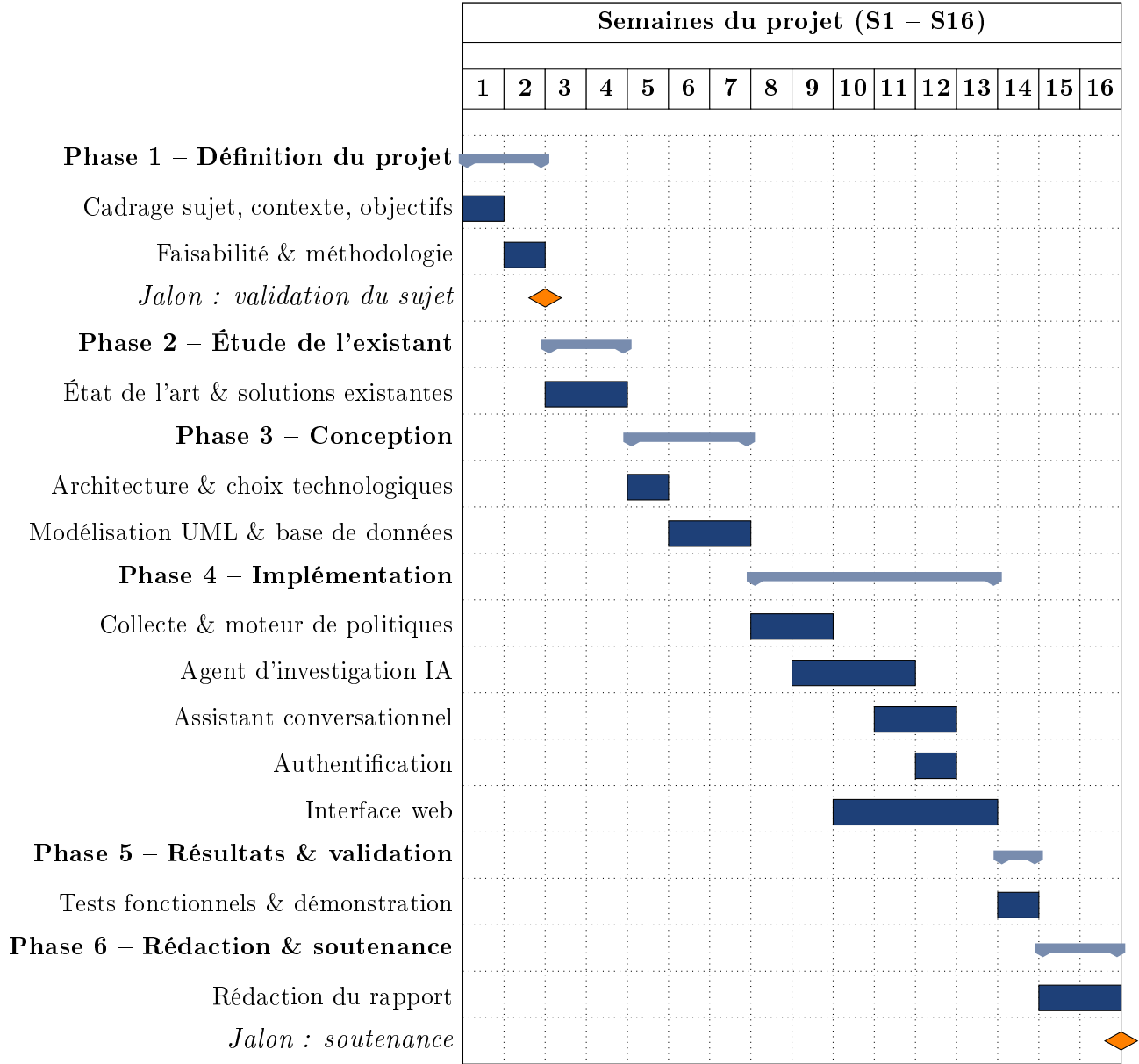


Figure 1.1: Diagramme de Gantt du projet

1.5 Étude de faisabilité

La faisabilité du projet a été évaluée selon trois axes :

Faisabilité technique L'ensemble des briques technologiques retenues (Wazuh, Python/-FastAPI, MySQL, Vue.js, API compatibles OpenAI pour les LLM) est **open source ou disponible en accès libre**, et documenté. Un environnement Wazuh de test est disponible ainsi qu'un accès à des API de LLM (Groq, compatible OpenAI) pour le prototypage rapide, avec la possibilité de migrer vers un modèle hébergé localement (Ollama) pour s'affranchir des contraintes de quota et de coût — cette portabilité est un objectif de conception du projet (voir chapitre 3).

Faisabilité temporelle Le découpage en phases courtes (figure 1.1) et la démarche

incrémentale permettent de disposer en permanence d'une version fonctionnelle, réduisant le risque de ne pas aboutir à une démonstration exploitable en fin de projet.

Faisabilité humaine Les compétences requises (développement backend Python, développement frontend JavaScript/Vue, notions de bases de données relationnelles, notions de sécurité informatique) correspondent au programme de la filière et sont réparties entre les membres du groupe.

1.6 Plan du rapport

Ce rapport est organisé en six chapitres. Le **chapitre 2** dresse un état de l'art des SIEM, des approches d'automatisation du triage d'alertes (SOAR) et des travaux récents sur l'usage d'agents fondés sur des modèles de langage en cybersécurité. Le **chapitre 3** présente la conception de la solution : architecture globale, modélisation UML et modèle de données. Le **chapitre 4** détaille l'implémentation : environnement de travail, traitement des données et description des modules réalisés. Le **chapitre 5** présente les résultats obtenus et leur discussion. Enfin, le **chapitre 6** conclut le rapport et ouvre sur les perspectives d'évolution du projet.

Chapter 2

État de l'art

2.1 Gestion des incidents de sécurité et SIEM

La gestion des incidents de sécurité informatique s'appuie sur un cycle largement standardisé dans l'industrie : préparation, détection et analyse, confinement/éradication/récupération, puis retour d'expérience. Ce cycle est notamment formalisé par le *National Institute of Standards and Technology* (NIST) dans son guide *Computer Security Incident Handling Guide* [1], qui reste une référence de facto pour la structuration des opérations de sécurité.

Les plateformes **SIEM** automatisent la partie « détection et analyse » en centralisant les journaux d'événements de multiples sources (systèmes d'exploitation, pare-feux, applications) et en appliquant des règles de corrélation pour générer des alertes. Elles constituent aujourd'hui un composant central de tout **SOC**, mais elles ne résolvent pas, par elles-mêmes, le problème du **volume** d'alertes à traiter.

2.2 Wazuh comme plateforme SIEM/XDR open source

Wazuh [2] est une plateforme de sécurité open source issue d'un *fork* d'OSSEC, qui combine les fonctions d'un *Host-based Intrusion Detection System* (HIDS), d'un collecteur de journaux et d'un moteur de règles de corrélation. Son architecture repose sur trois composants principaux :

- le **Wazuh Manager**, qui reçoit les événements des agents, applique les règles de détection et gère la configuration ;
- le **Wazuh Indexer**, un moteur de recherche et d'indexation basé sur **OpenSearch** [3] (un projet dérivé d'Elasticsearch), qui stocke les alertes et permet leur recherche performante ;
- le **Wazuh Dashboard**, une interface web de visualisation.

Ce projet s'appuie exclusivement sur le **Wazuh Indexer** comme source de données : les alertes y sont interrogées via son API REST compatible OpenSearch, ce qui découple totalement notre plateforme du Wazuh Manager et du Dashboard.

2.3 Fatigue d'alerte et automatisation (SOAR)

La littérature professionnelle en sécurité opérationnelle documente de longue date le phénomène de *alert fatigue* : les enquêtes annuelles menées auprès des équipes SOC (par exemple celles du SANS Institute) montrent de manière récurrente qu'une fraction importante des alertes générées ne sont jamais examinées faute de temps, et qu'une proportion significative des alertes traitées s'avèrent être des faux positifs. Ce constat a motivé l'émergence de la catégorie des outils **SOAR** (*Security Orchestration, Automation and Response*), dont des solutions commerciales comme Splunk SOAR, Microsoft Sentinel ou IBM QRadar SOAR sont des représentants connus. Ces outils automatisent des *playbooks* — des séquences d'actions prédéfinies (enrichissement, notification, blocage) — mais restent généralement limités à des règles déterministes : ils ne *raisonnent* pas sur le contexte d'une alerte au sens où le ferait un analyste humain.

2.4 Agents fondés sur les grands modèles de langage

L'émergence des **grands modèles de langage** (*Large Language Models*, LLM) a ouvert la voie à une nouvelle génération d'automatisation, fondée non plus sur des règles fixes mais sur un raisonnement itératif en langage naturel combiné à l'appel d'outils externes. Deux travaux fondateurs éclairent la conception retenue dans ce projet :

- Wei *et al.* [4] montrent que inciter un modèle à expliciter ses étapes de raisonnement (*chain-of-thought*) avant de produire une réponse améliore significativement la qualité de ses conclusions sur des tâches complexes ;
- Yao *et al.* [5] proposent le paradigme **ReAct** (*Reasoning + Acting*), dans lequel un modèle alterne entre raisonnement explicite et appels d'outils externes (recherche d'information, exécution d'actions), le résultat de chaque action venant enrichir le contexte du raisonnement suivant. Ce paradigme correspond très directement à la **boucle agentique** implémentée par l'*AI Investigator* de ce projet (chapitre 4) : le modèle observe une alerte, décide d'appeler un outil (recherche d'alertes corrélées, réputation d'une IP, etc.), lit le résultat, et itère jusqu'à pouvoir formuler un verdict.

Le référentiel **MITRE ATT&CK** [6], qui catalogue les tactiques et techniques employées par les attaquants, constitue par ailleurs une base de connaissance couramment mobilisée — bien qu'elle ne soit pas directement intégrée dans la version actuelle du projet — pour qualifier et prioriser les verdicts produits par ce type d'agent.

2.5 Comparatif des approches existantes

Le tableau 2.1 positionne Cyber Copilot par rapport aux grandes catégories d'outils existants.

Table 2.1: Comparatif entre les approches existantes et Cyber Copilot

Critère	SIEM seul (ex. Wazuh natif)	SOAR à play-books	Cyber Copilot
Détection d'alertes	Oui (règles)	Non (en amont)	S'appuie sur Wazuh
Triage automatique	Non	Oui (déterministe)	Oui (raisonnement IA)
Justification du verdict	Non	Limitée (log d'actions)	Explicite, tracée par action
Adaptation à un cas non prévu	Non	Non (règles figées)	Oui (raisonnement contextuel)
Interrogation en langage naturel	Non	Non	Oui (assistant conversationnel)
Coût / complexité de mise en œuvre	Faible	Élevé (licences)	Faible (open source + API/LLM local)

2.6 Synthèse et positionnement du projet

L'état de l'art fait apparaître un espace encore peu occupé par les solutions **open source** entre, d'un côté, les SIEM qui détectent mais ne concluent pas, et de l'autre, les SOAR commerciaux, coûteux, dont l'automatisation reste déterministe et peu adaptable. Cyber Copilot se positionne précisément dans cet espace : il conserve Wazuh comme moteur de détection éprouvé, et y ajoute une couche de raisonnement fondée sur un agent LLM inspiré du paradigme ReAct, avec une exigence forte de **traçabilité** (chaque action de l'agent est enregistrée) et de **sobriété** des accès aux données (l'agent n'a, par construction, aucun accès aux données d'identité des utilisateurs — voir chapitre 3).

Chapter 3

Conception

3.1 Architecture globale du système

La figure 3.1 présente l'architecture générale de Cyber Copilot. Le **Poller Service** interroge périodiquement le Wazuh Indexer pour récupérer les nouvelles alertes ; chaque alerte est évaluée par le **Policy Engine**, qui décide si elle mérite l'ouverture d'une investigation. Le cas échéant, l'**Investigation Manager** crée l'investigation (et associe/crée automatiquement l'actif concerné), puis la confie à l'**AI Investigator**, qui exécute sa boucle agentique en dialoguant avec un fournisseur de LLM et en appelant les outils nécessaires. Toutes les actions, preuves et le verdict final sont enregistrés en base MySQL. L'**API FastAPI** expose l'ensemble (investigations, politiques, actifs, assistant conversationnel, authentification) au **tableau de bord Vue.js**.

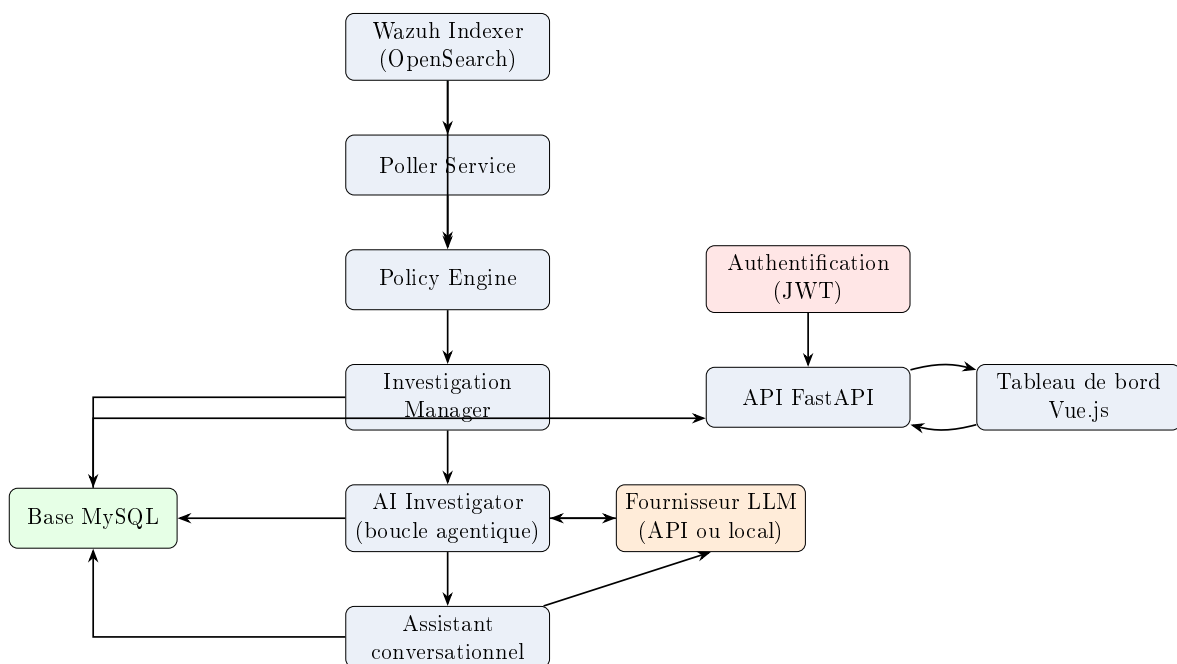


Figure 3.1: Architecture globale de Cyber Copilot

3.2 Diagramme de cas d'utilisation

Le figure 3.2 identifie les trois acteurs du système : l' **Analyste** (utilisateur humain authentifié), le **Poller** (déclencheur automatique périodique), et l'**Agent IA** (exécuté en tâche de fond pour le compte du système).

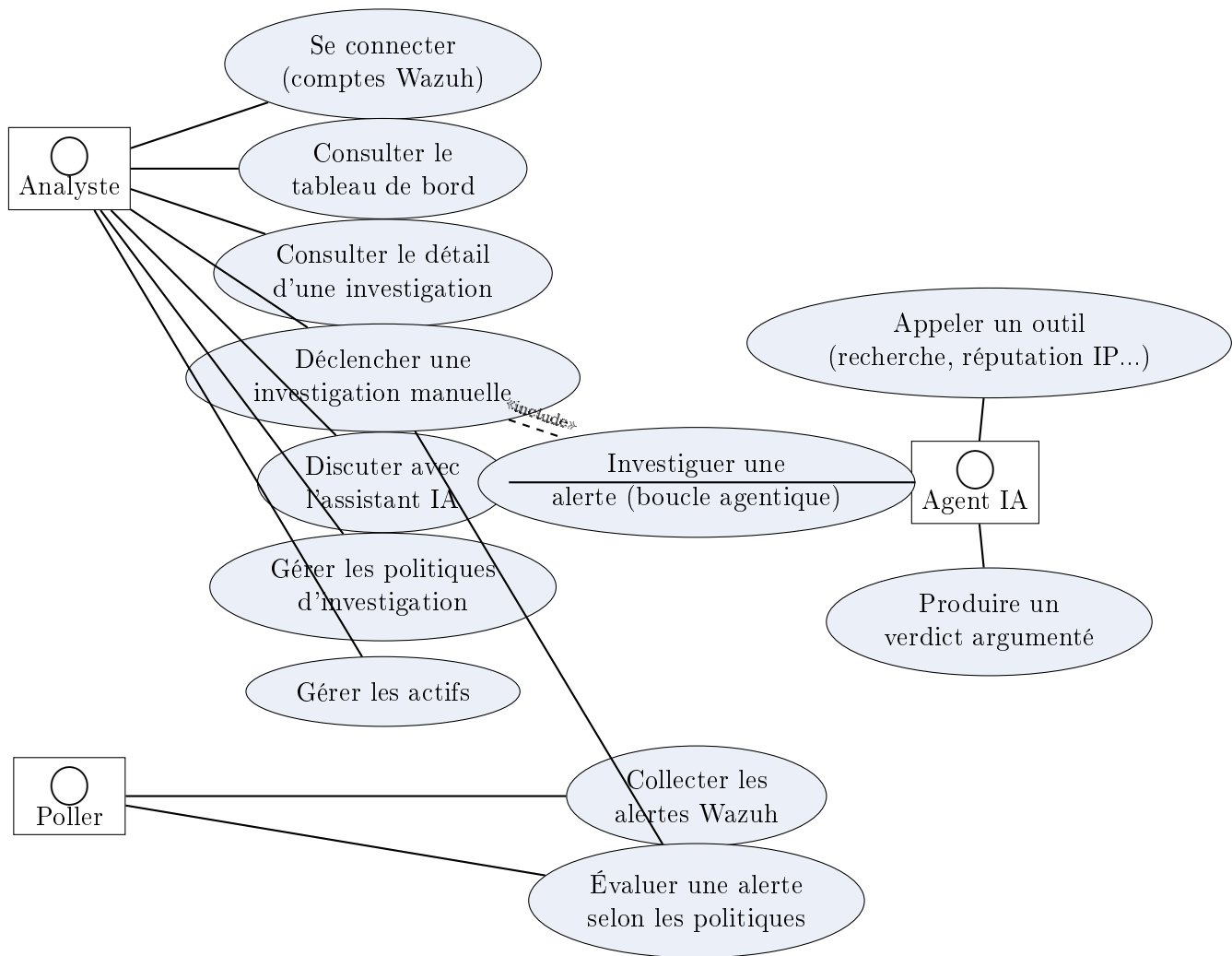


Figure 3.2: Diagramme de cas d'utilisation

3.3 Diagramme de classes

Le figure 3.3 présente un extrait représentatif du modèle de classes : la hiérarchie des `Tool` (chaque outil que l'IA peut appeler), la hiérarchie des `LLMProvider` (abstraction indépendante du fournisseur de modèle), et leur utilisation par les deux points d'entrée de raisonnement, `AIInvestigator` et `SecurityChatbot`.

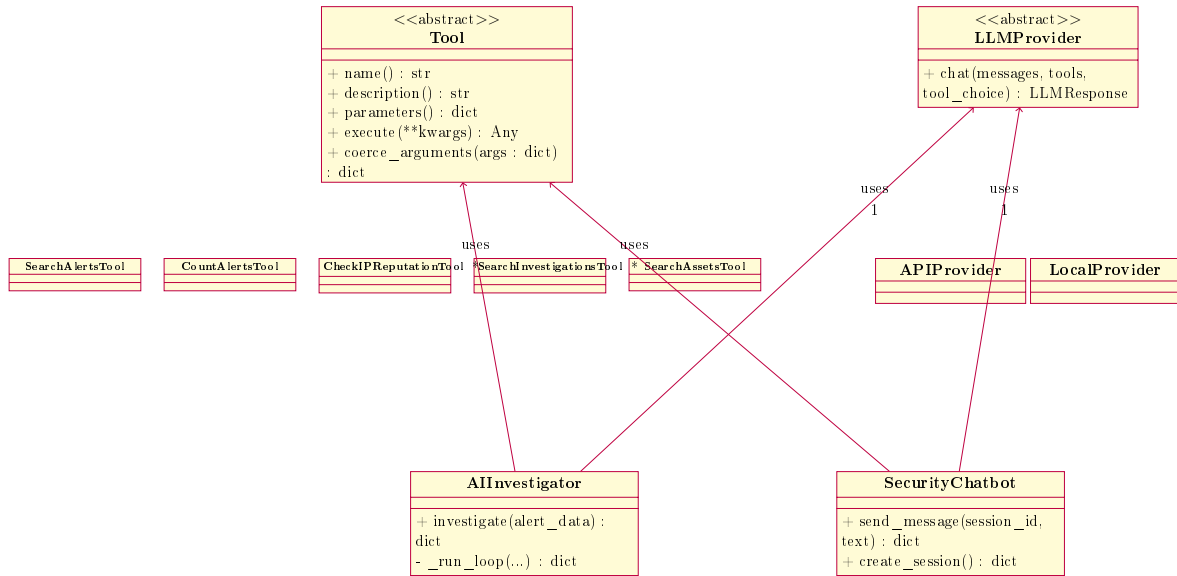


Figure 3.3: Diagramme de classes (extrait représentatif)

3.4 Diagrammes de séquence

3.4.1 Investigation automatique d'une alerte

Le figure 3.4 illustre le déroulement d'une investigation déclenchée automatiquement par le *Poller Service*, jusqu'à l'obtention d'un verdict par la boucle agentique.

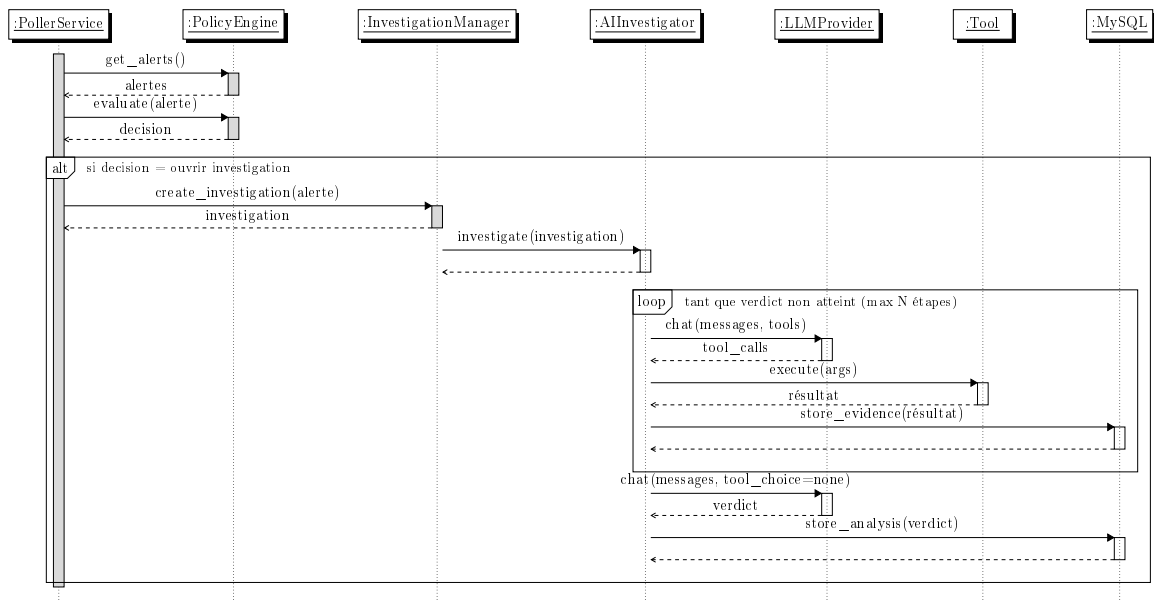


Figure 3.4: Séquence – investigation automatique d'une alerte

3.4.2 Authentification d'un utilisateur

Le figure 3.5 illustre le mécanisme d'authentification, fondé sur une délégation de la vérification des identifiants au Wazuh Indexer plutôt que sur une base de mots de passe

propre à l'application (voir section 3.7).

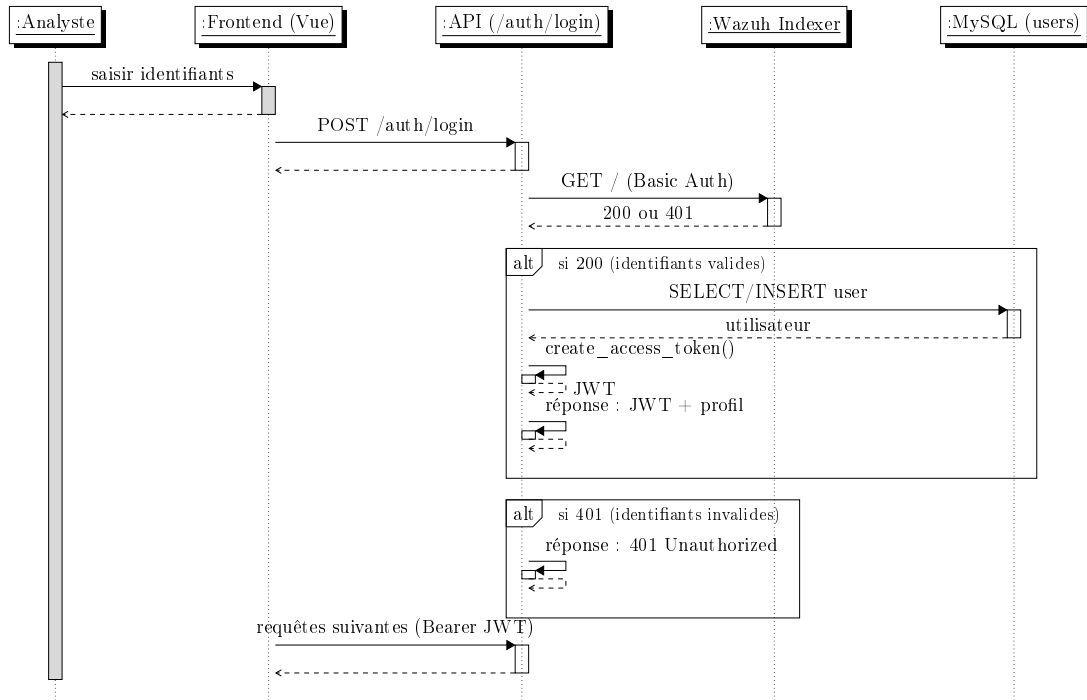


Figure 3.5: Séquence – authentification d'un utilisateur

3.5 Modèle de données

Le figure 3.6 présente le modèle entité-association simplifié de la base MySQL de la plateforme.

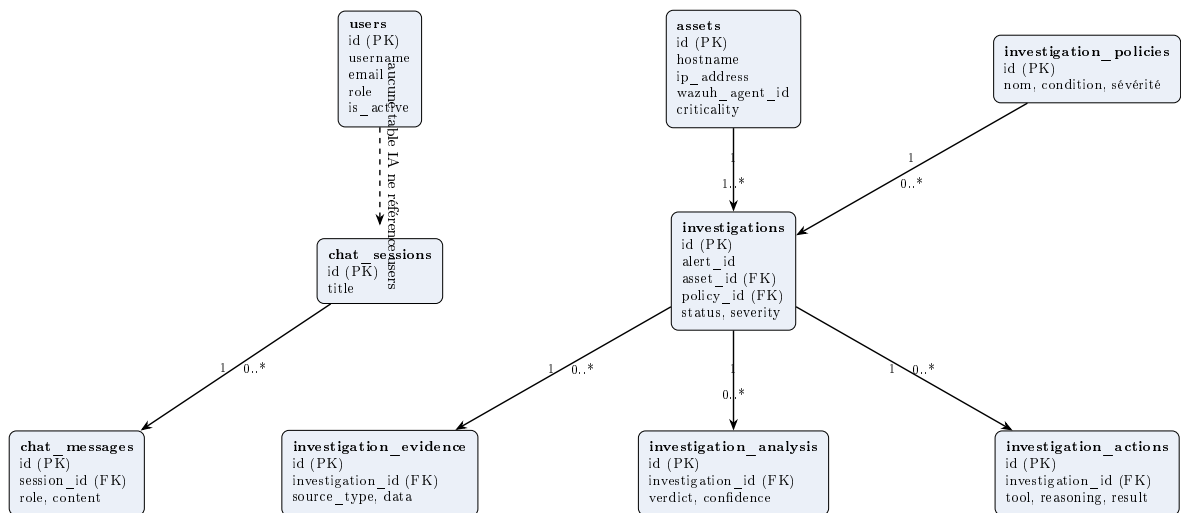


Figure 3.6: Modèle de données simplifié (MySQL)

Remarque de conception importante : la table **users** n'est **jamais** interrogée par les outils exposés à l'IA (ni pour l'agent d'investigation, ni pour l'assistant conversation-

nel). Cette exclusion n'est pas un filtre applicatif contournable : il n'existe tout simplement **aucune classe Tool** permettant de lire cette table (voir section 3.7).

3.6 Choix technologiques

Le tableau 3.1 justifie les principaux choix technologiques.

Table 3.1: Choix technologiques et justifications

Couche	Technologie	Justification
Backend	Python, FastAPI	Typage, performance asynchrone, génération automatique de documentation OpenAPI (/docs)
Accès données	SQLAlchemy / PyMySQL	Requêtes SQL explicites, contrôle fin adapté à un schéma relativement simple
Base applicative	MySQL	Fiabilité, transactions ACID, disponibilité large
Source d'alertes	Wazuh Indexer (OpenSearch), opensearch-py	Réutilisation du moteur de recherche déjà déployé avec Wazuh
IA / LLM	Interface LLMProvider abstraite (implémentations API compatible OpenAI et Ollama local)	Portabilité entre fournisseurs distants (Groq, OpenAI...) et modèle auto-hébergé, sans changement de code métier
Authentification	JWT + délégation au Wazuh Indexer	Évite la duplication d'une base de mots de passe (section 3.7)
Frontend	Vue 3 (Composition API), Pinia, Vue Router, Tailwind CSS, Vite	Écosystème léger, réactivité, rapidité de développement

3.7 Sécurité de la conception

Deux décisions de conception structurent la sécurité de la plateforme :

1. **Aucune base de mots de passe propre à l'application.** L'authentification (figure 3.5) délègue la vérification des identifiants au Wazuh Indexer : le mot de passe saisi n'est transmis qu'une seule fois, pour cette vérification, et n'est jamais stocké. Un jeton **JWT** signé, à durée de vie limitée, est ensuite émis par l'application pour authentifier les requêtes suivantes.
2. **Exclusion structurelle des données d'identité pour l'IA.** Aucun outil (Tool) exposé à l'agent d'investigation ou à l'assistant conversationnel ne permet

d’interroger la table `users`. Il ne s’agit pas d’un filtrage de requête, mais d’une absence totale de chemin d’accès : quelle que soit la formulation de la demande d’un utilisateur au chatbot, aucun outil disponible ne peut retourner de données d’identité.

Par ailleurs, la boucle agentique elle-même intègre plusieurs *garde-fous* destinés à borner son comportement (nombre maximal d’étapes, taille maximale du contexte renvoyé au modèle, détection des appels d’outils dupliqués, délais d’expiration réseau) — ces mécanismes sont détaillés au chapitre 4.

Chapter 4

Implémentation / Réalisation

4.1 Environnement de travail

4.1.1 Environnement logiciel

- **Langage backend** : Python 3.10
- **Framework web** : FastAPI [7], serveur ASGI Uvicorn
- **Base de données applicative** : MySQL 8
- **Client Wazuh Indexer** : opensearch-py
- **Frontend** : Vue 3 [8] (Composition API), Pinia (gestion d'état), Vue Router, Tailwind CSS, Vite (*build tool*)
- **Fournisseurs de LLM** : toute API compatible `/chat/completions` au format OpenAI [9] (Groq utilisé en développement), ou un modèle auto-hébergé via Ollama [10]
- **Enrichissement externe** : API AbuseIPDB [12] pour la réputation d'adresses IP
- **Gestion de versions** : Git

4.1.2 Environnement matériel

Le développement et les tests ont été réalisés sur une machine de développement standard (architecture x86-64, système Linux), le Wazuh Indexer et la base MySQL étant déployés localement pour les besoins du développement. Le caractère *stateless* de l'API et l'absence de traitement lourd côté serveur (le calcul est délégué au fournisseur de LLM) rendent le déploiement peu exigeant en ressources ; un serveur disposant de quelques gigaoctets de mémoire suffit pour l'exécution de l'API, de la base de données et, le cas échéant, d'un modèle local de taille modeste via Ollama.

4.2 Description et traitement des données

Deux sources de données coexistent dans la plateforme :

Les alertes Wazuh sont des documents JSON stockés dans le Wazuh Indexer, structurés notamment autour des champs `rule` (identifiant et niveau de sévérité de la

règle déclenchée), **agent** (identifiant et nom de l'hôte concerné), **data** (champs extraits par les décodeurs Wazuh, tels que `srcip/dstip`) et `full_log` (la ligne de journal brute). Ces documents sont interrogés en lecture seule ; la plateforme ne modifie jamais les alertes elles-mêmes.

Les **données applicatives** sont stockées dans une base MySQL relationnelle (schéma détaillé en figure 3.6), couvrant les investigations, les preuves collectées, les analyses produites, les actions de l'IA, les actifs, les politiques, les utilisateurs et les conversations de l'assistant. Les tables sont créées et migrées automatiquement au démarrage du serveur.

4.3 Modules principaux

4.3.1 Collecte et politiques (`poller_service.py`, `investigation_policy.py`)

Le **Poller Service** interroge le Wazuh Indexer à intervalle configurable, en ne retenant que les alertes postérieures au dernier horodatage traité (mécanisme de *watermark* persistant, pour éviter les doublons ou les pertes entre deux cycles). Chaque alerte récupérée est soumise au **Policy Engine**, qui évalue un ensemble de règles configurables (sévérité minimale, groupes de règles, agents concernés) afin de décider si une investigation doit être ouverte.

4.3.2 Gestion des investigations (`investigation_manager.py`)

L'**Investigation Manager** crée l'enregistrement d'investigation, recherche ou crée automatiquement l'**actif** correspondant à l'agent Wazuh concerné, puis transmet l'investigation à l'AI Investigator.

4.3.3 Agent d'investigation (`ai_investigator/investigator.py`)

C'est le cœur du projet. Étant donné une alerte déclenchante, l'agent :

1. récupère un **contexte de référence** (fiche de l'actif concerné et historique récent d'alertes du même agent) avant même de commencer à raisonner, pour répondre immédiatement à la question « ce comportement est-il habituel pour cet hôte ? » sans appel d'outil supplémentaire ;
2. engage une **boucle agentique** (inspirée du paradigme ReAct [5], chapitre 2) : à chaque étape, le modèle de langage reçoit l'historique de la conversation et la liste des outils disponibles, et peut soit appeler un outil, soit rendre son verdict ;
3. lorsqu'un outil est appelé, celui-ci est exécuté, son résultat est journalisé (table `investigation_actions`, avec le raisonnement du modèle au moment de l'appel) et réinjecté dans la conversation ;

4. la boucle se termine lorsque le modèle produit un verdict, ou lorsqu'un budget d'étapes est atteint, auquel cas un verdict final est forcé (`tool_choice="none"`) sur la base des preuves déjà réunies.

Garde-fous de la boucle agentique. Un modèle de langage n'étant pas un composant strictement déterministe ni intrinsèquement borné en coût, plusieurs mécanismes de protection ont été mis en place :

- un nombre maximal d'itérations (`MAX_STEPS`) ;
- une taille maximale par résultat d'outil et pour la conversation totale renvoyée au modèle (`MAX_TOOL_RESULT_CHARS`, `MAX_CONVERSATION_CHARS`), les appels *chat completions* étant *stateless* et donc facturés/limités sur l'intégralité du contexte renvoyé à chaque tour ;
- une **détection des appels d'outils dupliqués** : un appel strictement identique (même outil, mêmes arguments) à un appel déjà effectué dans la même investigation n'est pas réexécuté, son résultat déjà connu est simplement rappelé au modèle ;
- une **normalisation des arguments** reçus du modèle : un LLM peut renvoyer un entier sous forme de chaîne de caractères ("50" au lieu de 50) dans son appel de fonction ; chaque outil convertit automatiquement ses arguments selon son propre schéma JSON avant exécution (liste 4.1) ;
- des **délais d'expiration réseau** explicites sur les appels au fournisseur de LLM et au Wazuh Indexer ;
- une exécution de l'investigation entièrement encadrée par une gestion d'erreurs : toute exception (LLM, outil, base de données) aboutit à un verdict `ERROR` enregistré plutôt qu'à une investigation bloquée indéfiniment à l'état « en cours ».

```

1 def coerce_arguments(self, arguments: dict) -> dict:
2     """Cast argument values to match this tool's declared JSON
3     Schema types, e.g. {"limit": "50"} -> {"limit": 50}."""
4     props = self.parameters.get("properties", {})
5     coerced = dict(arguments)
6     for key, spec in props.items():
7         if key not in coerced or not isinstance(coerced[key], str):
8             continue
9         expected = spec.get("type")
10        raw = coerced[key].strip()
11        try:
12            if expected == "integer":
13                coerced[key] = int(raw)
14            elif expected == "number":
15                coerced[key] = float(raw)
16        except ValueError:
17            pass # left as-is; execute() surfaces the problem

```



```
18 return coerced
```

Listing 4.1: Normalisation générique des arguments d’un outil selon son schéma JSON

4.3.4 Outils disponibles pour l’IA

Le tableau 4.1 liste les outils partagés par l’agent d’investigation et l’assistant conversationnel. Ils sont volontairement en **lecture seule**.

Table 4.1: Outils exposés à l’IA

Outil	Source	Rôle
search_alerts	Wazuh Indexer	Rechercher des alertes par agent, IP, règle, plage temporelle, texte libre
count_alerts	Wazuh Indexer	Compter exactement les alertes correspondant à des filtres, sans les limites de <code>search_alerts</code>
get_alert_details	Wazuh Indexer	Détail complet d’une alerte
search_investigations	MySQL	Rechercher des investigations passées
count_investigations	MySQL	Compter exactement les investigations correspondant à des filtres
get_investigation,	MySQL	Détail, preuves et analyses d’une investigation
get_investigation_evidence,		
get_investigation_analysis		
search_assets,	MySQL	Rechercher/consulter un actif (hôte)
get_asset		
check_ip_reputation	AbuseIPDB [12]	Score de réputation d’une adresse IP publique (score de confiance d’abus, nombre de signalements, pays, fournisseur)

4.3.5 Assistant conversationnel (ai_investigator/chatbot.py)

L’assistant conversationnel réutilise exactement le même ensemble d’outils que l’agent d’investigation (figure 3.3), mais dans un cadre multi-tours en libre échange plutôt que centré sur une alerte unique. Pour éviter que la taille de la conversation — et donc le coût des appels au LLM — ne croisse indéfiniment au fil d’une session de discussion, seule la question de l’utilisateur et la réponse finale de l’assistant sont persistées : l’éventuel aller-retour d’appels d’outils au sein d’un tour de conversation est reconstruit à partir de l’historique, puis abandonné une fois la réponse produite.

4.3.6 Authentification (api/auth.py)

Conformément à la conception présentée en section 3.7, la vérification des identifiants (liste 4.2) consiste en une simple requête HTTP authentifiée vers le Wazuh Indexer :

une réponse 200 confirme la validité du couple identifiant/mot de passe, une réponse 401/403 l'infirme. Le mot de passe n'est jamais persisté.

```

1 def verify_wazuh_credentials(username, password, *,
2                               host, port, verify_ssl,
3                               timeout=5.0) -> bool:
4     try:
5         resp = requests.get(
6             f"https://{host}:{port}/",
7             auth=(username, password),
8             verify=verify_ssl,
9             timeout=timeout,
10        )
11    except requests.exceptions.RequestException:
12        return False
13    if resp.status_code == 200:
14        return True
15    if resp.status_code in (401, 403):
16        return False
17    return False # erreur cote Indexer, pas une preuve d'
invalidite

```

Listing 4.2: Vérification des identifiants auprès du Wazuh Indexer

Un jeton JWT signé (durée de vie configurable, 12 heures par défaut) est ensuite émis et doit accompagner chaque requête suivante (*header Authorization: Bearer <token>*). Toutes les routes de l'API sont protégées par ce mécanisme, à l'exception de la route de connexion elle-même et du point de contrôle de santé (*health check*).

4.4 Interface utilisateur

L'interface web (Vue 3) propose les écrans suivants : un **tableau de bord** synthétisant l'état des investigations en cours, une **liste des investigations** filtrable, une **vue détail** par investigation (chronologie des actions de l'IA, preuves, verdict), une **gestion des politiques** et des **actifs**, un **assistant conversationnel** (historique de sessions, affichage du raisonnement et de la trace des outils appelés lorsqu'ils sont disponibles), et un **écran de connexion**.

[Emplacement réservé — Capture d'écran – Tableau de bord]

*[Emplacement réservé — Capture d'écran – Détail d'une investigation
(chronologie des actions de l'IA et verdict)]*

[Emplacement réservé — Capture d'écran – Assistant conversationnel]

Chapter 5

Présentation des résultats

5.1 Scénario de bout en bout

Le scénario suivant illustre le fonctionnement complet de la chaîne mise en œuvre :

1. Un agent Wazuh détecte un événement suspect sur un hôte surveillé (par exemple, une création de compte utilisateur inhabituelle) et génère une alerte, indexée dans le Wazuh Indexer.
2. Le *Poller Service*, lors de son cycle suivant, récupère cette alerte et la soumet au *Policy Engine*, qui juge sa sévérité suffisante pour ouvrir une investigation.
3. L'*Investigation Manager* crée l'investigation, l'associe à l'actif correspondant, et la transmet à l'*AI Investigator*.
4. L'agent récupère son contexte de référence (fiche de l'actif, alertes récentes du même agent), puis décide, par exemple, d'interroger `search_alerts` pour vérifier si des alertes similaires ont déjà été observées sur ce même hôte, et `check_ip_reputation` si une adresse IP externe est impliquée.
5. Sur la base des preuves réunies, l'agent produit un verdict (`TRUE_POSITIVE`, `FALSE_POSITIVE` ou `NEEDS_REVIEW`) accompagné d'un niveau de confiance et d'une explication.
6. L'analyste consulte ce verdict, la chronologie complète des actions effectuées par l'IA et les preuves associées depuis l'interface web, et peut poser des questions complémentaires à l'assistant conversationnel (par exemple : « combien d'alertes ce mois-ci proviennent de cet hôte ? », désormais résolu de manière exacte par l'outil `count_alerts` plutôt qu'estimé à partir de résultats de recherche plafonnés).

5.2 Validations fonctionnelles effectuées

Le projet ne dispose pas, à ce stade, d'une suite de tests automatisés (voir section 5.3). La validation s'est appuyée sur des vérifications fonctionnelles ciblées, effectuées à chaque étape de développement contre les services réels (Wazuh Indexer, base MySQL, API de LLM), plutôt que sur des données simulées. Le tableau 5.1 en résume les principales.

Table 5.1: Principales validations fonctionnelles effectuées

Fonctionnalité validée	Méthode et résultat
Boucle d'investigation agentique	Exécution de bout en bout sur des alertes réelles jusqu'à l'obtention d'un verdict enregistré
Détection des appels d'outils dupliqués	Vérifié qu'un second appel identique réutilise le résultat mis en cache au lieu de solliciter à nouveau Wazuh/MySQL
Normalisation des arguments d'outils	Reproduction du cas où le modèle transmet un entier sous forme de chaîne (" limit ": "50"), confirmée corrigée après implémentation
Comptage exact des alertes/investigations	Comparaison entre le résultat de count_alerts/count_investigations et une requête de comptage directe : résultats identiques
Authentification	Scénarios : identifiants valides, identifiants invalides, jeton absent, jeton expiré/invalidé – chacun renvoie le code HTTP attendu (200 ou 401)
Délégation au Wazuh Indexer	Connexion réussie avec un compte Wazuh existant, rejet d'un mot de passe erroné
Restriction d'accès aux données d'identité	Confirmation qu'aucun outil disponible pour l'IA ne référence la table users
Réputation d'IP (AbuseIPDB)	Appel réel sur une IP publique connue et sur une IP privée (court-circuitée localement, sans appel réseau)
Interface web	Compilation de production du frontend sans erreur, navigation entre les écrans principaux

5.3 Limites constatées

- **Absence de suite de tests automatisés** : les validations décrites ci-dessus ont été effectuées manuellement à chaque évolution significative ; l'ajout de tests automatisés (unitaires sur les outils et la boucle agentique, tests d'intégration sur l'API) constitue une priorité pour la suite du projet.
- **Un seul niveau d'accès** : tout utilisateur authentifié dispose actuellement des mêmes droits sur l'ensemble des fonctionnalités ; la table **users** porte un champ de rôle exploitable, mais aucune vérification de permission n'est encore appliquée dessus.
- **Pas d'annulation d'une investigation en cours** : une fois démarrée, une investigation s'exécute jusqu'à son terme, une erreur, ou un redémarrage du serveur.
- **Variabilité liée au modèle de langage employé** : la qualité et la stabilité

des verdicts dépendent du modèle choisi ; les modèles de plus petite taille peuvent nécessiter davantage d'itérations avant de converger, ce que les garde-fous (chapitre 4) atténuent sans l'éliminer totalement.

- **Portée volontairement limitée des sources de données** : l'intégration d'autres sources (Suricata, journaux *cloud* de type CloudTrail) et de mécanismes de *retrieval-augmented generation* (RAG) sur une base de connaissances de menaces n'a pas été réalisée dans cette version, l'architecture par outils (**Tool**) étant conçue pour rendre ces ajouts additifs.

5.4 Discussion

L'approche agentique retenue présente un avantage déterminant par rapport à une automatisation à base de règles fixes : elle s'adapte à des situations non anticipées lors de la conception, en mobilisant les outils pertinents au cas par cas plutôt qu'en suivant un scénario figé. Cette flexibilité a toutefois un coût : la latence et le coût monétaire d'une investigation dépendent directement du nombre d'appels au modèle de langage, ce qui a directement motivé plusieurs des garde-fous mis en œuvre (budget d'étapes, détection de doublons, bornage de la taille du contexte). L'expérience du projet montre par ailleurs qu'un fournisseur de LLM externe introduit des contraintes opérationnelles propres (quotas de débit, formats de réponse parfois non strictement conformes au schéma attendu) qui ont dû être absorbées par la couche applicative plutôt que supposées absentes — la normalisation des arguments d'outils (liste 4.1) en est un exemple direct. C'est également ce qui justifie la conception de l'interface **LLMProvider** comme une abstraction remplaçable : le projet reste en mesure de migrer vers un modèle auto-hébergé sans remise en cause de la logique métier.

Chapter 6

Conclusion générale et perspectives

6.1 Synthèse du travail réalisé

Ce projet a permis de concevoir et développer Cyber Copilot, une plateforme qui étend une installation Wazuh existante d'une couche d'investigation automatisée fondée sur un agent d'intelligence artificielle. Le travail réalisé couvre l'ensemble de la chaîne : collecte périodique des alertes, moteur de politiques configurable, boucle agentique de raisonnement outillé inspirée du paradigme ReAct [5], traçabilité complète des actions de l'IA, assistant conversationnel partageant les mêmes outils, enrichissement par une source de renseignement externe (réputation d'IP), et authentification déléguée aux comptes Wazuh existants. L'ensemble a été validé fonctionnellement contre des services réels tout au long du développement (chapitre 5).

6.2 Difficultés rencontrées

Plusieurs difficultés ont jalonné le développement, et ont chacune donné lieu à une évolution de conception :

- les quotas de débit (*tokens per minute*) imposés par le fournisseur de LLM utilisé en développement se sont révélés plus contraignants que prévu, obligeant à réduire la taille du contexte transmis à chaque appel (invite système, contexte de référence, résultats d'outils) sans dégrader la qualité du raisonnement ;
- certains modèles ont ponctuellement tenté d'appeler un outil alors qu'il leur était explicitement demandé de conclure (`tool_choice="none"`), ce qui a nécessité la détection et la gestion explicite de ce cas plutôt que de le traiter comme une erreur fatale ;
- les arguments transmis par le modèle lors d'un appel d'outil ne respectent pas toujours strictement le type attendu (un nombre transmis sous forme de texte), ce qui a conduit à une normalisation systématique des arguments avant exécution (liste 4.1) ;
- la conception d'un mécanisme d'authentification sans dupliquer la base de comptes existante a demandé une réflexion spécifique, résolue par la délégation de la vérification au Wazuh Indexer.

Ces difficultés, bien que ponctuellement contraignantes, ont eu un effet positif sur

la robustesse finale du système : chacune a été traitée comme un cas à absorber structurellement plutôt que comme une exception isolée.

6.3 Perspectives

Plusieurs axes d'évolution ont été identifiés pour la suite du projet :

- **Gestion fine des permissions** : exploiter le champ de rôle déjà présent sur les utilisateurs pour restreindre certaines actions (par exemple la gestion des politiques) aux seuls profils autorisés ;
- **Migration vers un modèle de langage auto-hébergé** : l'abstraction `LLMProvider` a précisément été conçue pour permettre cette transition sans modification de la logique métier, afin de s'affranchir des contraintes de quota et de coût d'une API distante ;
- **Élargissement des sources de données** : intégration d'autres capteurs (Suri-cata pour la détection réseau, journaux *cloud* de type CloudTrail) sous forme de nouveaux outils, sans remise en cause de l'architecture ;
- **Enrichissement par RAG** : adossement de l'agent à une base de connaissances de menaces (par exemple structurée autour du référentiel MITRE ATT&CK [6]) pour améliorer la qualification des verdicts ;
- **Suite de tests automatisés** : couverture unitaire des outils et de la boucle agentique, et tests d'intégration de l'API, pour sécuriser les évolutions futures ;
- **Possibilité d'annuler une investigation en cours, et tableaux de bord analytiques** (tendances de charge d'alertes, taux de faux positifs observé dans le temps).

En définitive, ce projet démontre la faisabilité d'une automatisation du triage d'alertes de sécurité par un agent fondé sur un modèle de langage, tout en conservant l'exigence de traçabilité et de sobriété d'accès aux données propre à un contexte de cybersécurité — les deux axes qui ont guidé l'ensemble des choix de conception présentés dans ce rapport.

Références

- [1] P. Cichonski, T. Millar, T. Grance, K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, 2012.
- [2] Wazuh Inc., *Wazuh Documentation*, <https://documentation.wazuh.com/>, consulté en 2026.
- [3] OpenSearch Project, *OpenSearch Documentation*, <https://opensearch.org/docs/>, consulté en 2026.
- [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, arXiv:2201.11903, 2022.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv:2210.03629, 2022.
- [6] MITRE Corporation, *MITRE ATT&CK Framework*, <https://attack.mitre.org/>, consulté en 2026.
- [7] S. Ramírez, *FastAPI Documentation*, <https://fastapi.tiangolo.com/>, consulté en 2026.
- [8] Vue.js Core Team, *Vue.js Guide*, <https://vuejs.org/guide/>, consulté en 2026.
- [9] OpenAI, *Function calling — API Reference*, <https://platform.openai.com/docs/guides/function-calling>, consulté en 2026.
- [10] Ollama, *Ollama Documentation*, <https://ollama.com>, consulté en 2026.
- [11] OWASP Foundation, *OWASP Top 10*, <https://owasp.org/www-project-top-ten/>, consulté en 2026.
- [12] AbuseIPDB, *AbuseIPDB API Documentation*, <https://docs.abuseipdb.com/>, consulté en 2026.

Appendix A

Annexes

A.1 Structure du dépôt de code

1	alert_receiver.py	Client Wazuh Indexer (recherche/lecture d'alertes)
2	poller_service.py	Boucle de collecte periodique, watermark
3	investigation_policy.py	Moteur de politiques (quelles alertes ouvrir)
4	investigation_manager.py	Creation des investigations, liaison des actifs
5	database.py	Schema MySQL (DDL + migrations) + acces donnees
6		
7	ai_investigator/	
8	investigator.py	Boucle agentique (agent d'investigation)
9	chatbot.py	Assistant conversationnel (memes outils)
10	llm/	Interface LLMPProvider (API / local)
11	tools/	Outils Wazuh + base de donnees +
	renseignement	
12		
13	api/	
14	main.py	Application FastAPI, cycle de vie,
	routes	
15	dependencies.py	Cablage des services (DB, poller, IA,
	LLM)	
16	auth.py	Verification des identifiants + JWT
17	routers/	auth, investigations, politiques, assets
	,	
18		users, system (poller), chat
19		
20	frontend/	
21	src/views/	Tableau de bord, Investigations, Actifs
	,	
22		Politiques, Parametres, Chat,
	Connexion	
23	src/stores/	Etat Pinia (investigations, chat, auth
	)	
24	src/api/	Client HTTP + modules par ressource

Listing A.1: Arborescence simplifiée du projet

A.2 Extrait du schéma de la table investigations

```
1 CREATE TABLE IF NOT EXISTS investigations (  
2     id          CHAR(36)          PRIMARY KEY,  
3     alert_id    VARCHAR(255) NOT NULL,  
4     asset_id    CHAR(36),  
5     policy_id   CHAR(36),  
6     title       VARCHAR(255),  
7     description TEXT,  
8     status      VARCHAR(20) NOT NULL DEFAULT 'QUEUED',  
9     severity    VARCHAR(20),  
10    closed_at   DATETIME,  
11    created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
12    updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
13              ON UPDATE CURRENT_TIMESTAMP,  
14    FOREIGN KEY (asset_id) REFERENCES assets(id),  
15    FOREIGN KEY (policy_id) REFERENCES investigation_policies(id)  
16 );
```

Listing A.2: Extrait de définition de la table investigations

A.3 Extrait de l'invite système de l'agent d'investigation

```
1 You are an expert AI security analyst working on a Security  
2 Investigation Platform.  
3  
4 Task: investigate a security alert and determine TRUE POSITIVE,  
5 FALSE POSITIVE, or NEEDS REVIEW.  
6  
7 ## Process  
8 1. Read the triggering alert AND the baseline context below it  
9   (asset record + recent alert history for the same agent).  
10 2. Only call tools for what's still missing.  
11 3. Correlate the evidence, then give your verdict.  
12  
13 ## Be efficient - budget matters as much as accuracy  
14 - Reach a verdict in 2-4 tool calls for most alerts.  
15 - Duplicate calls (same tool + same arguments) are blocked.  
16 - ...
```

Listing A.3: Extrait du *system prompt* de l'AI Investigator

A.4 Variables de configuration principales (.env)

Table A.1: Principales variables d'environnement

Variable	Rôle
WAZUH_INDEXER_HOST/PORT/ USERNAME/PASSWORD	Connexion au Wazuh Indexer (compte de service applicatif)
MYSQL_HOST/PORT/USER/ PASSWORD/DATABASE	Connexion à la base applicative
LLM_PROVIDER	local (Ollama) ou api (tout point de terminaison compatible OpenAI)
LLM_MODEL, LLM_API_KEY	Modèle utilisé et clé d'accès le cas échéant
LLM_MAX_TOKENS, LLM_REASONING_EFFORT	Bornes de génération et effort de raisonnement (selon le modèle)
ABUSEIPDB_API_KEY	Active l'outil <code>check_ip_reputation</code> si renseignée
AUTH_JWT_SECRET, AUTH_TOKEN_TTL_MINUTES	Signature et durée de vie des jetons de session

A.5 Glossaire complémentaire

Agent (Wazuh) Logiciel installé sur un hôte surveillé, chargé de collecter les événements et de les transmettre au Wazuh Manager.

Alerte Événement de sécurité généré lorsqu'une règle de détection est déclenchée par un événement collecté.

Boucle agentique Cycle itératif dans lequel un modèle de langage alterne raisonnement et appels d'outils jusqu'à produire une conclusion.

Faux positif Alerte ne correspondant à aucune menace réelle.

Verdict Conclusion produite par l'agent d'investigation (`TRUE_POSITIVE`, `FALSE_POSITIVE`, `NEEDS_REVIEW`), avec un niveau de confiance et une justification.